# 31. Mutexes, Atomics, and sync Primitives

## 31. Mutexes, Atomics, and sync Primitives

**Phase:** Phase 5 — Concurrency
**Estimated time:** 1–2 days
**Slug:** `31-mutexes-atomics`

### Learning objectives

- Protect shared memory with mutexes
- Use RWMutex for read-heavy maps
- Know atomic operations for simple counters
- Prefer confining data to one goroutine when possible

### Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

### Why this matters

Mutexes protect shared memory. Atomics handle simple counters without heavy locks. Choosing wrongly causes races or slowdowns.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

### Core concepts

- sync.Mutex: Lock/Unlock around critical sections.
- RWMutex for read-heavy workloads.
- sync/atomic for integers and pointers with care.
- Never copy a struct containing a Mutex after first use.
- `go test -race` is mandatory for concurrent code.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

### Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-mutexes, && cd /tmp/gw-mutexes,
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "fmt"
    "sync"
)

func main() {
    var mu sync.Mutex
    var n int
    var wg sync.WaitGroup
    for i := 0; i < 1000; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            mu.Lock()
            n++
            mu.Unlock()
        }()
    }
    wg.Wait()
    fmt.Println(n)
}
```

Run it:

```
go run .
```

Expected output:

```
1000
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

## Mini-lab (`go run` steps)

1. Increment without mutex; run -race.
2. Fix with Mutex; confirm race gone.
3. Replace with atomic.AddInt64 and compare.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

## Curriculum notes (expanded from the compact draft)

## Mutex

```
type Safe struct {
    mu sync.Mutex
    m  map[string]int
}

func (s *Safe) Inc(k string) {
```

```
    s.mu.Lock()
    defer s.mu.Unlock()
    s.m[k]++
}
```

## Atomics

`atomic.Int64` for counters without mutex — not a free pass for complex invariants.

## Design preference

1. Don't share (owning goroutine)
2. Channels for ownership transfer
3. Mutex for shared structures
4. Atomics for simple stats

Always `go test -race`.

## Failure modes to watch while coding

- Copying a struct containing a mutex
- Holding locks across slow I/O
- Assuming map access is safe concurrently

When a test fails, ask which failure mode you hit before changing code at random.

## Idioms vs Python pitfalls

- threading.Lock ≈ Mutex.
- GIL can mask races — Go will not.
- Prefer channels when communicating ownership; mutex when sharing state.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

## Going deeper

Keep critical sections tiny. Do I/O outside the lock.

## Practice checklist

Before the exercise, confirm you can:

1. Explain "Mutexes, Atomics, and sync Primitives" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

## Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

## Common mistakes (especially from Python instincts)

- Copying a struct containing a mutex
- Holding locks across slow I/O
- Assuming map access is safe concurrently

## Exercises

Work in `exercises/31-mutexes-atomics/`. Read that folder's `README.md` before you code.

### What you will build

Protect shared counters/state so tests pass under race detector.

### Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

**Do not open `solutions/31-mutexes-atomics/` until you have an honest failing attempt.**

## Self-check

Answer without scrolling up:

1. Why defer Unlock?
2. RWMutex when?
3. Why race detector with mutex code?

## Further reading

- https://pkg.go.dev/sync
- https://pkg.go.dev/sync/atomic